Laboratorul 4 - Producer / Consumer Problem

Introducere

Scopul laboratorului

Scopul laboratorului este de a implementa problema *Producer / Consumer* printr-o problema care calculeaza punctajele la un concurs, determinand clasamentul , folosind syncronizare in Java.

Trebuia implementat fie in Java fie in C++. Am ales Java.

Restrictii:

- nu folosim variabile conditionale , ci folosim mecanismul de wait si notify
- sincronizarea se face la nivel de lista (in cadrul laboratorului 4, blocam intreaga lista cand adaugam sau stergem elemente)
- lista se tine ordonata si este o lista simplu inlantuita
- un concurent poate avea tentative de frauda si astfel e eliminat din concurs

Abordare

- am implementat 2 variante:

1. Secventiala
    - am lista simpla inlantuita
    - un set de concurenti fraudulosi
    - trec prin fiecare fisier cu problemele la fiecare tari si adaug (sau updatez scorul) in lista de concurenti (daca nu au fost gasiti fraudulosi) si daca au vreun scor semanalat ca fiind fraudulos (-1) ii adaug in setul de concurenti fraudulosi si ii sterg din lista inlantuita daca exista
    - la final salvez in fisierul de output clasamentul
2. Paralela
    - am o lista simpla inlantuita cu rezultate
    - un set de concurenti fraudulosi
    - o coada creata de mine in care pun de la threaduriile readere liniile citite care merg la threadurii-le workers care le proceseaza
    - un object lock
    - lista de fisiere unde trimit pentru fiecare thread lista de fisiere pe care le proceseaza (le impart in mod egal si daca nu se impart in mod egal trimit cate un fisier in plus la thread ul urmator pana nu mai am rest de fisiere)
    - *Pornesc threaduriile de readers*:
        - fiecare thread pargurge fisierele primite si pune in coada cate o lini , facand notify la lock

        - la final dupa ce se parcurg thread uriile este cerut ca unul dintre ele sa scrie rezultatele in fisier astfel ca la threadul de reader mai trimit un parametru ''writeToFile'' care indica daca threadul trebuie sa scrie in fisier sau nu.
        - daca threadul trebuie sa scrie incep si parcurg lista de rezultate si scriu in fisier

    - *Pornesc threadurile de workers*:

        - fiecare thread worker primeste linii din coada si le proceseaza
        - daca linia este de tipul ''-1 -1'' inseamna ca nu mai sunt linii de citit si threadul worker se opreste Procesare: - folosesc un while infinit care se opreste cand citeste linia ''-1 -1'' - fac un lock si cat timp coada este empty astept - cand nu mai e scot si verific daca linia este de tipul ''-1 -1'' , daca nu o procesez Procesare linie: - impart linia in id si score. - pun un syncronized pe lista de rezultate - verific daca scorul este -1 si daca este il adaug in setul de concurenti fraudulosi si il sterg din lista de rezultate - daca nu este fraudulos il adaug in lista de rezultate

    - *Pun in coada -1 -1* :

        - fac asta pentru a semnala ca nu mai sunt linii de citit si threadurile workers sa se opreasca (la fiecare thread worker)

    - *Scriu in fisier*:

        - asa cum am spus primul thread reader care trebuie sa scrie in fisier primeste un parametru ''writeToFile'' care indica daca trebuie sa scrie in fisier sau nu

    - *Verific daca rezultatul este corect*:

        - folosesc doua dictionare pentru a verifica daca rezultatul coincide cu cel secvential
        - daca nu coincide afisez un mesaj de eroare

Complexitate spatiu

- O(n) - unde n este numarul de concurenti

Tehnologii folosite

- Java:
    - syncronized
    - threaduri
    - wait and notify
    - map and set

Teste rulate

- structura fisierelor de test este cum a fost mentionat in enunt:

```
    5 tari cu cate maxim 100 de concurenti si minim 80 de concurenti in fiecare
tara si 10 probleme – nu este acelasi numar de concurenti in fiecare tara!
    Generarea datelor se va face random cu specificatia ca:
    - Probabilitatea de a nu se rezolva o problema sa fie maxim 10% pe fiecare
```

```
tara
    - Probabilitatea de a fi frauda sa fie maxim 2%
```

Teste: a. secvential b. paralel cu p_w =4,6,8.16 si p_r=1 c. paralel cu p_w =4,6,8.16 si p_r=2 Masurarea timpului de executie este in nanosecunde. Se masoara atat citirea cat si scrierea in fisier si procesarea datelor.

Date de intrare

- pentru fiecare tara (5) avem 10 fisiere (reprezentant problemele) si rezultate concurentiilor la aceste probleme

Structura de date

- lista simplu inlantuita pentru concurenti
- set pentru concurentii fraudulosi
- coada pentru a trimite linii de la threadurile de reader la threadurile de worker
- lock pentru a face syncronized pe lista de rezultate

Partitionare pe threaduri

- threadurile de reader parcurg fisierele si pun in coada linii
- threadurile de worker iau linii din coada si le proceseaza (nu stim in ce ordine vin liniile si astfel nu mai avem problema cu schedulerul)

Clasele

1. **Clase principale**

Clasa `Main`

- **Rol**: Controlul principal al aplicației.
- **Membri**:
  - `String type` - Tipul execuției (`sequential` sau `parallel`).
  - `int nrOfReaderThreads` - Numărul de thread-uri pentru cititori.
  - `int nrOfWriterThreads` - Numărul de thread-uri pentru lucrători.
- **Funcții**:
  - `main(String[] args)` - Inițializează tipul execuției și gestionează logica principală, inclusiv măsurarea timpului.

Clasa `Parallel`

- **Rol**: Implementarea metodei paralele pentru prelucrarea datelor.
- **Membri**:
  - `CustomQueue<String> queue` - Coada utilizată pentru comunicarea între thread-uri.
  - `LinkedList result` - Rezultatele finale ordonate.
  - `Set<String> blackList` - Lista participanților eliminați pentru fraude.
- **Funcții**:

- `run(int nrOfReaderThreads, int nrOfWriterThreads)` - Inițializează thread-urile cititorilor și lucrătorilor, coordonează execuția și gestionează resursele.

Clasa `Sequential`

- **Rol**: Implementarea metodei secvențiale pentru prelucrarea datelor.
- **Membri**:
  - `LinkedList list` - Rezultatele finale ordonate.
  - `Set<String> blackList` - Lista participanților eliminați pentru fraude.
- **Funcții**:
  - `run()` - Procesează toate fișierele în ordine și generează rezultatul final.

---

2. **Clase Utilitare**

Clasa `Constants`

- **Rol**: Gestionarea constantelor globale ale aplicației.
- **Membri**:
  - `int NO_OF_COUNTRIES` - Numărul total de țări.
  - `int NO_OF_PROBLEMS` - Numărul total de probleme.
  - `String OUTPUT_DIR` - Directorul unde se generează fișierele.

Clasa `Generator`

- **Rol**: Generarea de date de intrare pentru testare.
- **Membri**:
  - `int NO_OF_COUNTRIES`, `int NO_OF_PROBLEMS`, `int MAX_NO_OF_PARTICIPANTS`, `int MIN_NO_OF_PARTICIPANTS`, `double PROBABILITY_OF_FAILURE_PER_COUNTRY`, `double PROBABILITY_OF_FRAUD`.
  - `String OUTPUT_DIR` - Directorul unde se generează fișierele.
- **Funcții**:
  - `generate()` - Creează și scrie fișierele de intrare pentru toate țările și problemele.

Clasa `FileManagment`

- **Rol**: Gestionarea operațiilor de fișiere, inclusiv compararea fișierelor de ieșire.
- **Funcții**:
  - `areFilesEqual(String file1, String file2)` - Verifică dacă două fișiere sunt identice.

---

3. **Clase pentru Gestionarea Datelor**

Clasa `CustomQueue<T>`

- **Rol**: Implementarea unei cozi personalizate generice.
- **Membri**:
  - `Node<T> head`, `Node<T> tail` - Nodurile de început și sfârșit ale cozii.
  - `int size` - Dimensiunea cozii.

- **Funcții**:
  - `enqueue(T item)` - Adaugă un element în coadă.
  - `dequeue()` - Scoate un element din coadă.
  - `peek()`, `isEmpty()`, `size()` - Operații auxiliare.

Clasa `LinkedList`

- **Rol**: Implementarea unei liste ordonate de participanți.
- **Membri**:
  - `LinkedListE head` - Capul listei.
- **Funcții**:
  - `add(LinkedListE element)`, `addOrUpdate(LinkedListE element)` - Adaugă sau actualizează un element în listă.
  - `remove(String key)` - Elimină un participant.
  - `search(String key)` - Caută un participant în listă.

Clasa `LinkedListE`

- **Rol**: Reprezintă un element al listei.
- **Membri**:
  - `String participant` - ID-ul participantului.
  - `Integer score` - Scorul total al participantului.
  - `LinkedListE next` - Referința la următorul element.

---

4. **Clase pentru Thread-uri**

Clasa `ReaderThread`

- **Rol**: Citirea datelor din fișiere sau scrierea rezultatelor.
- **Membri**:
  - `List<String> files` - Lista fișierelor de procesat.
  - `CustomQueue<String> queue` - Coada unde sunt puse liniile citite.
  - `ILinkedList results` - Referință pentru salvarea rezultatelor.
  - `Boolean writeToFile` - Indicarea modului de operare (citire/scriere).
- **Funcții**:
  - `run()` - Execută logica de citire sau scriere.

Clasa `WorkerThread`

- **Rol**: Prelucrarea liniilor citite și actualizarea listei.
- **Membri**:
  - `CustomQueue<String> queue` - Coada cu liniile de procesat.
  - `LinkedList list` - Lista rezultatelor.
  - `Set<String> blackList` - Lista neagră a participanților.
- **Funcții**:
  - `run()` - Procesează liniile din coadă și actualizează lista rezultatelor.
  - `process(String line)` - Gestionează adăugarea sau eliminarea unui participant.

Analiza rezultatelor:

Type	NOOfReaderThreads	NOOfWriterThreads	Time
sequential	0	0	95.11702
parallel	1	4	50.59431
parallel	1	6	51.72124
parallel	1	8	52.32508
parallel	1	16	52.91029
parallel	2	4	50.51185
parallel	2	6	50.30872
parallel	2	8	54.39132
parallel	2	16	52.21977

Diferente:

- Implementarea secventiala este aproximativ de 2 ori mai lenta decat implementarea paralela.
- Putem observa ca cel putin pe masina mea implementariile paralele cu 4 si 6 threaduri de scriere sunt cele mai rapide.
- Exista si o mica diferenta cand vine vorba de numarul de threaduri de citire, dar aceasta diferenta este mult mai mica decat cea a threadurilor de scriere. Probabil daca erau mai multe fisiere de citit am fi observat o diferenta mai mare.
- Cand vorbim de 8,16 threaduri de scriere observam ca timpul de executie creste, probabil din cauza faptului ca se face mai multa sincronizare si mai multe operatii de citire si scriere in fisier.
- Astfel se fac mai multe lockuri si unlockuri si astfel creste timpul de executie.